

GUIA DE ONBOARDING

QA & Automação de Testes

Manual introdutório para pessoas que estão iniciando no setor de QA do Projeto Atual Mais, com foco em teste manual e automação com Playwright.

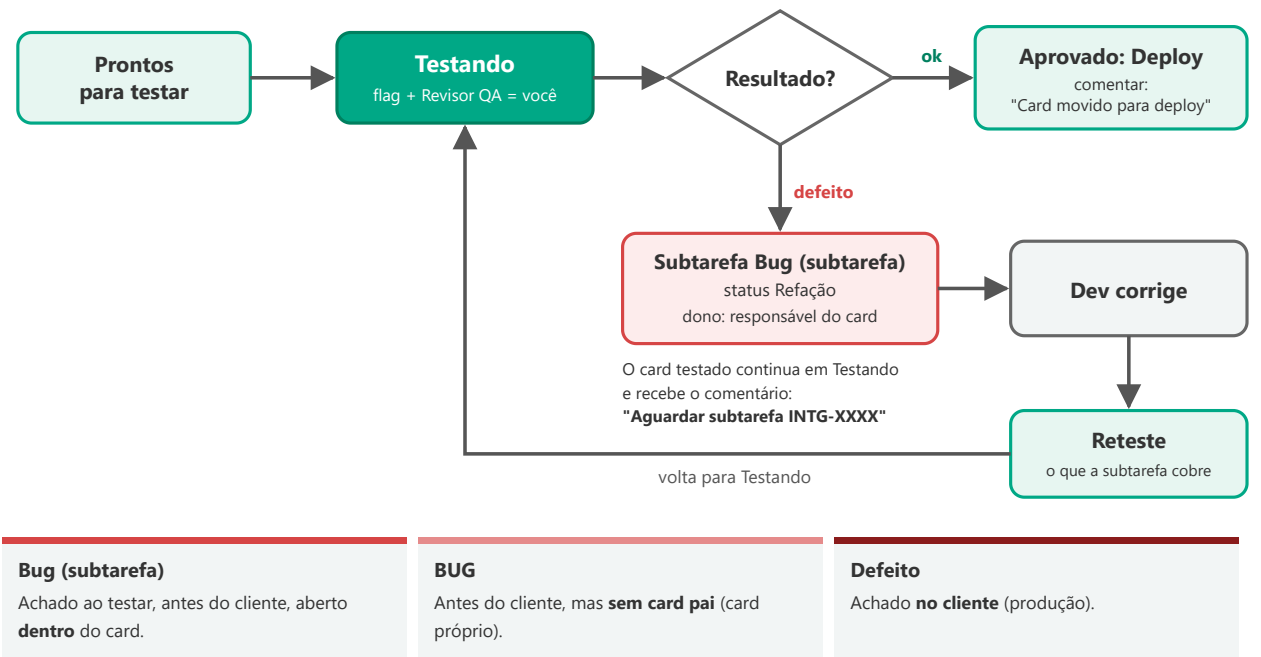
ATUAL SOLUÇÕES

Material interno · Setor de QA · Versão 1.11

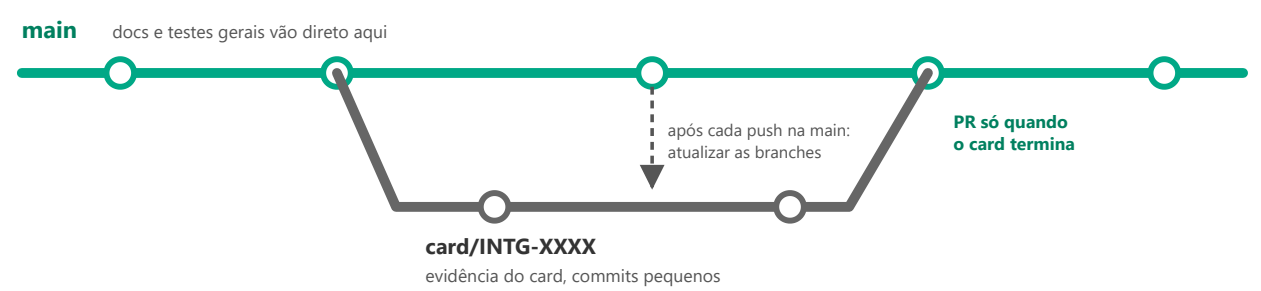
Mapa visual do setor de QA (leia primeiro)

Este mapa mostra o trabalho em 5 desenhos. Os detalhes estão nas seções seguintes, e a **fonte sempre atualizada** são os documentos da pasta **docs/** do repositório.

1. O caminho de um card no Jira



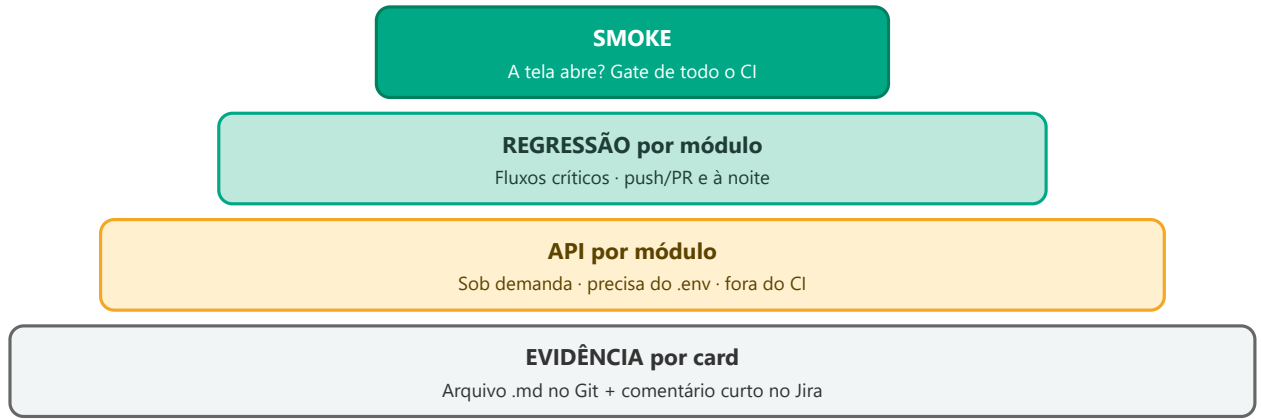
2. Como o Git é organizado



Cada card tem a sua branch. O que serve a todos (testes, manuais, modelos) vai na main.
Depois de cada push na main: confira o CI (aba Actions) e atualize as branches de card abertas.

Mapa visual: testes, pastas e CI

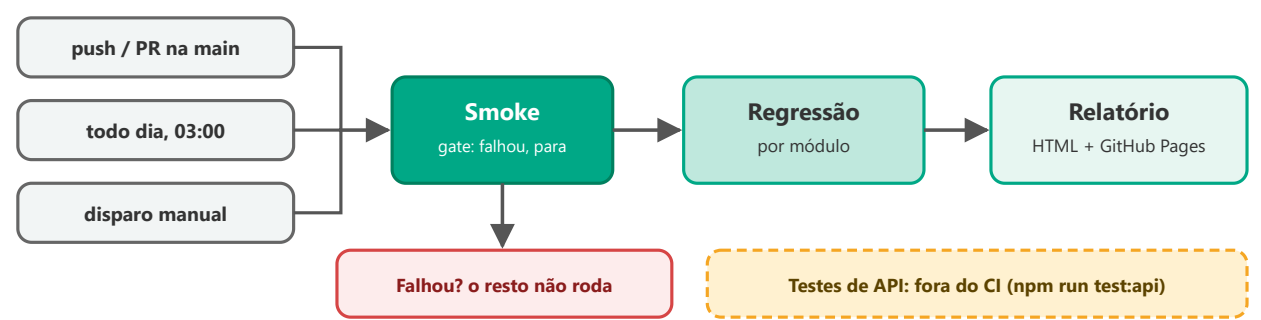
3. As camadas de teste



4. Onde cada coisa fica

tests/smoke/	A tela abre?	@smoke CI: sempre primeiro
tests/regression/<módulo>/<área>/	Regressão macro por módulo (vendas, financeiro)	@regression CI: push/PR e à noite
tests/api/<módulo>/<área>/	Contrato do backend (hoje: financeiro)	@api sob demanda, só em HMG
tests/support/	Login por API, ambiente e mapas de tela (pages/)	compartilhado por todos
tests/cards/	Evidência de cada card e os modelos	npm run check:evidencias
docs/	Mapa do sistema, Jira, Git, telas, falhas e API	fonte viva; este PDF resume

5. O que o CI faz



Regra de ouro: o Jira guarda a **decisão e o resumo**; o detalhe fica **versionado no Git**. Fato provado é separado de hipótese, e nada de senha, token ou .env no repositório.

1. Objetivo do manual

Este manual apresenta a estrutura básica utilizada pelo setor de QA e Automação de Testes do Projeto Atual Mais: os ambientes de teste, as ferramentas do dia a dia, o fluxo de trabalho entre teste manual e automatizado, e a forma esperada de documentar e registrar as atividades.

Objetivo principal: dar autonomia rápida a quem está entrando no setor, reduzindo a dependência de explicações verbais e reunindo em um só lugar o necessário para começar a testar e automatizar com segurança.

Versão em mapa mental: este mesmo guia também está disponível em formato visual/interativo em mm.tt/map/4095504042.

2. Visão geral do setor de QA

O time de QA é responsável por validar as entregas do Atual Mais antes que cheguem à produção, cobrindo tanto testes manuais quanto testes automatizados end-to-end com Playwright.

O trabalho envolve: analisar cards e bugs reportados, desenhar cenários de teste, validar manualmente nos ambientes apropriados, automatizar os fluxos mais críticos e recorrentes, e documentar resultados de forma que qualquer pessoa do time consiga reproduzir o que foi encontrado.

O Atual Mais é um sistema ERP web (frontend React/Ant Design, via Umi) que cobre operações comerciais, fiscais, financeiras e administrativas — por isso, testes de fluxos fiscais e de venda exigem atenção redobrada.

3. Ferramentas utilizadas

As ferramentas abaixo fazem parte da rotina do setor de QA e possuem funções específicas no processo de teste e automação:

Ferramenta	Finalidade
Jira	Gestão dos cards de teste, bugs encontrados e acompanhamento das entregas do time de QA.
Clockify	Registro do tempo dedicado a cada atividade de teste, manual ou automatizado.
Clarity	Gerenciamento e controle de acessos utilizados nos ambientes de teste.
GitHub	Versionamento: uma branch por card; docs e testes gerais direto na main; PR só quando o card termina.
GitHub Actions	Pipeline de CI: smoke primeiro, regressão em todo push/PR e suite completa à noite (sem os testes de API). Confirma o resultado depois de cada push.
VS Code	Editor utilizado para escrever, rodar e depurar os testes automatizados.
Playwright	Framework de automação de testes end-to-end utilizado pelo time de QA.
JAM	Extensão do navegador para gravar o passo a passo e evidenciar bugs encontrados.
DevTools (navegador)	Inspeção de elementos, seletores e chamadas de rede durante teste manual e automação.
Claude	Apoio na análise de HTML/seletores, escrita de testes e documentação das atividades.

Importante: nem toda automação substitui teste manual. Fluxos críticos (fiscal, pagamento, estoque) devem ser validados manualmente antes de serem automatizados.

4. Ambientes de teste

HMG: o ambiente padrão

Todo card é testado aqui primeiro. É onde roda a automação.

<https://backoffice.hmg.atualmais.com.br/>

PREPROD: operações críticas

Usado quando o card impacta vendas, entradas, tributos, orçamentos, OS e rotinas fiscais.

<https://backoffice.preprod.atualmais.com.br/>

Produção: nunca automatizar

Backoffice. Existe um link de teste em produção, para comparar comportamento.

<https://backoffice.atualmais.com.br/>

Link de teste:
<https://company8.atualmais.com.br/>

Regra prática: a suíte Playwright (local ou no CI) roda em **HMG** e nunca contra produção.

5. Preparar a empresa de teste (fiscal)

Antes de validar NFC-e ou NF-e, a empresa de teste no HMG precisa estar configurada. Faz-se **uma vez por ambiente**.

- 1 Solicitar o link do ambiente HMG, se ainda não tiver.
- 2 Em **Configurações > Editar > Dados Gerais**: preencher CNPJ e Inscrição Estadual (tabela 5.2).
- 3 Em **Configurações > Editar > Faturamento**: configurar NFC-e e NF-e (tabela 5.1).

5.1 Faturamento: NFC-e e NF-e lado a lado

Campo	NFC-e	NF-e
Tipo	65 - Nota Fiscal do Consumidor Eletrônica	55 - Nota Fiscal Eletrônica
Forma de Emissão	1 - Normal	1 - Normal
Ambiente	2 - Homologação	2 - Homologação
Tipo de impressão	0 - Geração de DANFE	1 - DANFE normal, Retrato
Equipamento Fiscal	não se aplica	1 - satDinamico
Série / Nota Fiscal	numeração alta (ex.: 511)	numeração alta (ex.: 511)
ID Token	000001	não se aplica
Token (CSC)	A8BEE050FF3601DE0531E5FA00A9D26	não se aplica
Versão do QRCode	2.0	não se aplica

5.2 Empresa de teste (Dados Gerais)

Campo	Valor
Razão Social	SG AUTOMAÇÃO COMERCIAL LTDA ME - ATUAL SGP
CNPJ	12.266.496/0001-29
IE	083.632.80-8

Cuidados com teste fiscal

- Sempre em **homologação**.
- Usar **séries altas**: as baixas podem estar em uso por outras pessoas.
- O **certificado digital** deve ser solicitado a um integrante do time; Token/ID e Token/CSC não o dispensam em alguns testes.

6. Fluxo de trabalho: do card ao veredito

- 1 Pegar o card em **Prontos para testar**, mudar a flag para **Testando** e se atribuir em **Revisor QA**. Conferir o número na URL.
- 2 Mapear os cenários: **feliz**, **negativo** e **borda** (seção 10).
- 3 Validar no **HMG** (PREPROD quando aplicável) e registrar o tempo no Clockify.
- 4 Registrar a evidência em **tests/cards/INTG-XXXX-evidencias.md** e comentar o veredito no card, em texto curto.
- 5 **Aprovado**: mover para deploy. **Defeito**: subtarefa **Bug (subtarefa)** em **Refação**, atribuída ao responsável do card; o card fica em Testando com "Aguardar subtarefa INTG-XXXX" (desenho 1).
- 6 Se o cenário for crítico ou recorrente, automatizar: tela em **tests/regression/<módulo>/<área>/**, API em **tests/api/<módulo>/<área>/**.
- 7 Git: branch **card/INTG-XXXX**; docs e testes gerais direto na **main**; PR só quando o card termina. Depois de um push na **main**, confira o CI (aba Actions) e atualize as branches de card abertas (desenho 2).

Fonte sempre atualizada: **docs/jira-fluxo-de-trabalho.md** e **docs/git-fluxo-de-trabalho.md**.

7. Projeto de automação (Playwright)

Repositório **github.com/romuloatual/atual_mais**. Regra de organização: **camada, módulo e área** (ex.: **regression/vendas/venda-rapida/**). As pastas estão no mapa visual (desenho 4) e os módulos e áreas do sistema em **docs/mapa-do-sistema.md**; a configuração global fica em **playwright.config.ts** (baseURL padrão = HMG).

Comandos do dia a dia

Para	Comando	O que faz
Rodar	npm test	a suíte, sem os testes de API
	npm run test:smoke	só a suíte de fumaça (@smoke)
	npm run test:regression	só a regressão (@regression)
	npm run test:api	testes de API (só em HMG, precisa do .env)
	npm run test:card -- @INTG-2645	só os testes de um card
Ver	npm run test:headed	roda com o navegador visível
	npm run test:ui	interface interativa do Playwright
	npm run test:report	abre o último relatório
Criar	npm run test:codegen	grava ações no navegador e gera código de teste
Conferir	npm run check:evidencias	confere se as evidências seguem o modelo
	npm run limpar:títulos	prévia da limpeza de títulos de teste no HMG

Antes de escrever um teste de tela novo, rode **npm run test:codegen** no fluxo que você quer automatizar: ele já gera boa parte dos seletores e das ações.

8. Integração contínua (CI)

O workflow `.github/workflows/playwright.yml` roda os testes no GitHub Actions. O **smoke** roda sempre primeiro: se falhar, o resto não roda (desenho 5).

Gatilho	Quando	O que roda depois do smoke
Push / PR na main	a cada envio	regressão (@regression)
Agendado	todo dia, 03:00 (Brasília)	suíte completa, sem os testes de API
Manual	quando você dispara	à escolha: só a regressão ou a suíte completa

API fica fora do CI Precisa de credenciais, e a quitação deixa registros que o sistema não exclui. Roda sob demanda: npm run test:api .	Relatório HTML por execução, guardado 14 dias e publicado em romuloatual.github.io/atual_mais .	Nunca produção O baseURL padrão é o HMG, o mesmo das validações manuais.
--	--	--

9. Boas práticas para seletores e testes

Prefira - Id (<code>#id</code>). - Role ou texto visível (<code>getByRole</code>, <code>getByText</code>). <code>data-testid</code>, quando existir.	Evite Classes de CSS Modules com hash (ex.: <code>customerName__AxNXm</code>): o hash muda a cada deploy e quebra o teste sem a funcionalidade ter mudado.
---	---

Ao mapear uma tela nova

- 1 Use o DevTools (**Copy > Copy outerHTML**) para pegar o HTML já renderizado: o app é uma SPA (Umi/React), então o HTML estático não basta.
- 2 Em modais, popups ou iframes (ex.: pré-visualização de impressão), confirme se o elemento está dentro de um **#document** aninhado.
- 3 Documente qualquer diferença entre o card e a tela: o bug pode já ter sido corrigido em parte, ou o escopo mudou.

10. Como documentar um caso de teste e um problema

Regra de ouro: o Jira guarda a decisão e o resumo; o detalhe fica **versionado no Git**. Texto enxuto, evidência à vontade (print, vídeo, payload).

Onde registrar cada coisa

O quê	Onde	Modelo (tests/cards/)
Evidência do card	tests/cards/INTG-XXXX-evidencias.md	TEMPLATE-evidencias.md
Veredito	comentário curto no card do Jira	TEMPLATE-comentario-jira.md
Defeito	card ou subtarefa no Jira	TEMPLATE-card-de-defeito.md
Caso de teste	dentro da evidência do card	TEMPLATE-caso-de-teste.md

Em cada card, cubra três tipos de cenário (1 de cada por campo ou regra)

Tipo	O que testa	Exemplo
Caminho feliz	o que deve dar certo	Telefone e Celular válidos aparecem no impresso
Negativo	dado inválido que deve ser barrado	letras em campo numérico não podem ser aceitas
Borda	estourar o limite	campo de 10 caracteres, testar com 11

Cenário N – [feliz / negativo / borda] [nome curto]
Dado [estado inicial e dados]
Quando [ação do usuário]
Então [resultado esperado, conforme o card]
Obtido: [o que aconteceu de fato] – passou / falhou
Evidência: [print, vídeo, payload ou PDF]

Ao encontrar um problema (Obtido diferente do Esperado)

- 1 Abrir no **modelo de defeito do time** (Resumo, Objetivo, Regra de Negócio, Gherkin, Notas técnicas). Tipo **Bug (subtarefa)** se for do card em teste; **BUG** sem card paj; **Defeito** se foi achado no cliente.
- 2 Informar o ambiente (HMG, PREPROD, Produção). Se já existia, comparar com a produção e escrever "pré-existente".
- 3 Anexar evidências: prints, payload sem token, vídeo no JAM; em teste automatizado, o relatório do Playwright ou o artefato do CI.
- 4 **Separar fato de hipótese:** "provado por API ou payload" é uma coisa; "parece que" é outra. Nunca colar CPF, token ou senha.

11. Checklist de entrada no setor de QA

- ☐ Acessar e conhecer o Jira do projeto.
- ☐ Configurar e utilizar o Clockify.
- ☐ Ter acesso ao Clarity.
- ☐ Ter acesso ao GitHub e clonar o repositório `github.com/romuloatual/atual_mais`.
- ☐ Ter o VS Code configurado, com a extensão Playwright Test for VSCode instalada.
- ☐ Instalar o Node.js e rodar `npm install` no projeto de automação.
- ☐ Rodar `npx playwright install` para instalar os navegadores de teste.
- ☐ Instalar/conhecer o JAM.
- ☐ Conhecer o uso do Claude como ferramenta de apoio.
- ☐ Conhecer os ambientes HMG, PREPROD e Produção.
- ☐ Conhecer a convenção de tags e de pastas (camada e módulo) e a pipeline de CI no GitHub Actions.
- ☐ Ler os documentos da pasta `docs/` do repositório (mapa do sistema, fluxo do Jira, fluxo do Git, telas, falhas e testes de API).
- ☐ Saber criar a branch do card, commitar, enviar e, depois de um push na main, conferir o CI e atualizar as branches abertas (`docs/git-fluxo-de-trabalho.md`).
- ☐ Para testes de API: criar o arquivo `.env` a partir do `.env.example` e preencher com o usuário de teste do HMG (o `.env` nunca vai para o Git).
- ☐ Conferir uma evidência de card com `npm run check:evidencias`.
- ☐ Solicitar acesso/certificado digital quando necessário para testes fiscais.
- ☐ Rodar a suíte de testes local pela primeira vez (`npm test`) e confirmar que passa.

12. Principais regras

- Todo teste, manual ou automatizado, deve estar vinculado a um card no Jira.
- O tempo das atividades deve ser registrado no Clockify.
- Testes automatizados (local ou CI) rodam por padrão contra HMG — nunca contra produção.
- Cada card tem a sua evidência em **tests/cards/INTG-XXXX-evidencias.md**; o teste automatizado que sobra só vai para **tests/regression/<módulo>/<área>/** depois de validado e considerado crítico/recorrente.
- O Jira guarda a decisão e o resumo; o detalhe fica versionado no Git. Fato provado é separado de hipótese.
- No Git: uma branch por card; **git add** pelo nome do arquivo (nunca **git add .** sem olhar); depois de cada push na **main**, conferir o CI e atualizar as branches de card abertas.
- Nunca versionar senha, token ou **.env**. Testes de API só rodam em HMG e criam dados de teste que o sistema pode não excluir.
- Seletores baseados em classes com hash (CSS Modules) devem ser evitados na automação.
- Erros devem ser documentados de forma clara e reproduzível, com evidências anexadas.
- Dados de teste fiscal devem ser usados somente nos ambientes apropriados.
- Materiais e documentos do setor devem seguir o padrão visual da Atual Soluções.

Mentalidade esperada: entender o fluxo antes de testar, validar manualmente antes de automatizar, e deixar evidências suficientes para que qualquer pessoa do time consiga reproduzir o que foi encontrado.

13. Identidade Visual — Atual Soluções

A identidade visual dos materiais do setor deve seguir o padrão institucional da Atual Soluções, mantendo consistência nas apresentações, documentos e comunicações internas.

Elemento	Especificação
Verde institucional	PANTONE Green C · CMYK 100/0/65/0 · RGB 0/168/134 · HEX #00A886
Cinza institucional	PANTONE Cool Gray 10 C · CMYK 0/0/0/72 · RGB 102/102/102 · HEX #666666
Tipografia	Família sem serifa, com hierarquia clara entre títulos, subtítulos e texto.
Estilo	Visual limpo, corporativo e tecnológico, com boa hierarquia de informação e uso consistente das cores institucionais.

Regra: as cores institucionais não devem ser alteradas arbitrariamente em materiais oficiais.